

ANÁLISIS DEL RETO

Juan Sebastián Rojas Marín, 202610140, js.rojasm123@uniandes.edu.co

Katy Teran Mercado, 202614247, k.teranm@uniandes.edu.co

Juan José Ricon Santos, 202615789, jj.rincons1@uniandes.edu.co

Requerimiento <<3>>

Descripción

```
def req_3(catalog, country, channel):  
    """  
    Retorna el resultado del requerimiento 3  
    """  
    # TODO: Modificar el requerimiento 3  
    start_time = get_time()  
  
    orders = catalog["orders"]  
    count = 0  
  
    buscar_prod = {}  
    buscar_ano = {}  
  
    result = {  
        "prom_box" : 0,  
        "prom_disc" : 0,  
        "prom_mark" : 0,  
        "prom_box_ship" : 0,  
        "prod_freq" : None,  
        "ano_freq" : None  
    }  
    for pos in range(len(orders)):  
        orden = orders[pos]  
  
        if country == orden["Country"] and channel == orden["Channel"]:  
            count += 1  
  
            result["prom_box"] += float(orden["Price_per_Box"])  
            result["prom_disc"] += float(orden["Discount_Pct"])  
            result["prom_mark"] += float(orden["Marketing_Spend"])  
            result["prom_box_ship"] += int(orden["Boxes_Shipped"])
```

```

        product = orden["Product"]
        year = orden["Order_Date"][:4]

        buscar_prod[product] = buscar_prod.get(product, 0) + 1
        buscar_ano[year] = buscar_ano.get(year, 0) + 1

    if count == 0:
        return None

    result["prom_box"] = result["prom_box"] / count
    result["prom_disc"] = result["prom_disc"] / count
    result["prom_mark"] = result["prom_mark"] / count
    result["prom_box_ship"] = result["prom_box_ship"] / count

    max_count = 0

    for llave, valor in buscar_prod.items():
        if valor > max_count:
            max_count = valor
            result["prod_frec"] = llave

    max_count = 0

    for llave, valor in buscar_ano.items():
        if valor > max_count:
            max_count = valor
            result["ano_frec"] = llave

```

```

end_time = get_time()
time = delta_time(start_time, end_time)

return time, count, result

```

Este requerimiento recibe un país y un canal de venta. A partir de esos datos, recorre los pedidos almacenados para encontrar los que cumplen ambas condiciones. Con los pedidos encontrados se calculan los promedios de Price_per_Box, Discount_Pct, Marketing_Spend y Boxes_Shipped. También se identifica el producto que aparece con mayor frecuencia y el año en el que se registró la mayor cantidad de pedidos.

Entrada	Catálogo de pedidos, Country y Channel.
Salidas	Tiempo de ejecución, cantidad de pedidos encontrados, promedio de Price_per_Box, Discount_Pct, Marketing_Spend y Boxes_Shipped, producto más frecuente y año con mayor cantidad de pedidos.
Implementado (Sí/No)	Sí. Implementado por Juan José Rincón

Análisis de complejidad

Análisis de complejidad de cada uno de los pasos del algoritmo

Pasos	Complejidad
Recorrer los n pedidos del ArrayList	$O(n)$
Verificar Country y Channel en cada pedido	$O(1)$
Actualizar sumas para los promedios	$O(1)$
Calcular los promedios finales	$O(1)$
Recorrer los productos encontrados para hallar el más frecuente	$O(p)$
Recorrer los años encontrados para hallar el más frecuente	$O(a)$
Construir el resultado	$O(1)$
TOTAL	$O(n)$

Análisis

El comportamiento de este requerimiento depende principalmente de la cantidad de pedidos cargados, ya que para saber cuáles corresponden al país y al canal ingresados es necesario revisar los elementos del ArrayList. En cada pedido se comparan Country y Channel y, cuando cumple ambas condiciones, se suman los valores necesarios para calcular los promedios. También se actualiza cuántas veces aparece cada producto y cada año. Estas operaciones se realizan directamente sobre cada pedido y tienen complejidad $O(1)$ en promedio.

Después del recorrido se calculan los promedios y se revisan las frecuencias guardadas para encontrar el producto y el año que más se repiten. Aunque estas últimas búsquedas dependen de la cantidad de productos y años diferentes, ambas cantidades están contenidas dentro de los pedidos ya revisados. Por eso, en el peor de los casos, la complejidad total del requerimiento se mantiene en $O(n)$, donde n es la cantidad de pedidos almacenados.

Requerimiento Ejemplo

Descripción

```
def get_data(data_structs, id):  
    """  
    Retorna un dato a partir de su ID  
    """  
    pos_data = lt.isPresent(data_structs["data"], id)  
    if pos_data > 0:  
        data = lt.getElement(data_structs["data"], pos_data)  
        return data  
    return None
```

Este requerimiento se encarga de retornar un dato de una lista dado su ID. Lo primero que hace es verificar si el elemento existe. Dado el caso que exista, retorna su posición, lo busca en la lista y lo retorna. De lo contrario, retorna None.

Entrada	Estructuras de datos del modelo, ID.
Salidas	El elemento con el ID dado, si no existe se retorna None
Implementado (Sí/No)	Si. Implementado por Juan Andrés Ariza

Análisis de complejidad

Análisis de complejidad de cada uno de los pasos del algoritmo

Pasos	Complejidad
Buscar si el elemento existe (isPresent)	$O(n)$
Obtener el elemento (getElement)	$O(1)$
TOTAL	$O(n)$

Análisis

A pesar de que obtener un elemento en un *ArrayList*, dada su posición, tiene complejidad constante, la implementación de este requerimiento tiene un orden lineal $O(n)$. Esto debido a que, lo primero que se hace es verificar si el elemento hace parte de la lista. Específicamente, a la hora de buscar un elemento en una lista, en el peor de los casos es necesario recorrer toda la lista, es decir, complejidad lineal.